

ALPACA AI TRADING AGENTS HACKATHON · 2026

Options Alpha Agent

An autonomous options agent that asks whether the market is paying enough for risk
— and refuses to trade when it isn't.

Paper account **PA3BAT100EFE**

Repo github.com/nizarhazaymeh/tradingbot

Team **Mahdi Anagreh · Nizar Hazaymeh · Ali**

THE PROBLEM

Predicting direction is the wrong question

Most trading agents ask which way the market will move. Nobody answers that reliably, and an LLM given a broad prompt cannot be audited when it is wrong.

WHAT GENERIC AGENTS DO

Hand the model a prompt and a broker key, let it choose instruments, strikes and sizes. When it loses you cannot tell whether the thesis or the execution failed.

WHAT OPTIONS ADD

Structural constraints a generic agent gets wrong: four-leg order caps, coverage rules, sign conventions on credit versus debit, and missing Greeks near expiry.

So we changed the question to one with a measurable answer: **is this specific structure priced better than the risk it carries?**

WHAT WE MEASURED FIRST

Zero of 60 spreads were **positive-EV**

We enumerated 60 real SPY/QQQ/IWM spreads and scored expected value using the market's own implied volatility. Not one was positive. That is not a bug — it is what an efficient market looks like.

0/60

positive-EV when priced with the market's own implied vol

5/12

clear a 2%-of-risk threshold when probability uses realised vol

VRP

the variance risk premium is the only honest source of edge

0

QQQ structures traded — its implied vol sat below realised

Under its own implied vol every vanilla spread prices at fair value minus the spread you cross. If the edge is real it has to come from somewhere the market is structurally mispricing.

Price with implied vol. Score probability with realised vol.

Implied volatility persistently exceeds subsequently realised volatility, because option sellers are paid to bear variance risk. The gap is the edge, and it appears honestly in the expected value.



+3.0% variance risk premium — the agent sells it. When the gap inverts, it trades nothing.

Selling options is like selling insurance. It only makes money when the premium charged is bigger than the real risk. The agent measures both, every cycle, per underlying.

The model approved trades that lost money by design

Our expected-value model priced every leg at mid and ignored the bid/ask. In the first live session, every position the agent opened had a round-trip spread *larger than the edge the gate demanded*.

STRUCTURE	RISK	EDGE REQUIRED	ROUND-TRIP SPREAD
iron condor SPY	\$441	\$8.82	\$36.00
iron condor QQQ	\$444	\$8.88	\$27.00
bear put IWM	\$356	\$7.11	\$9.00

\$68 of that session's \$89 loss was execution cost, not adverse movement. EV is now computed net of the spread crossed twice, and the optimiser responded by preferring narrower structures — cost scales with leg count, edge does not.

Thirteen defects surfaced only by trading live: stale-quote pricing that never filled, an exception between submission and persistence that left a position with no stop attached, a shared ledger that let one account read another's positions, and a profit target on directional trades that demanded a 4x return. All fixed, all covered by tests.

ARCHITECTURE

The model opines. **Code decides.**

Seven stages per underlying per cycle. Two use the LLM; five are deterministic. Everything that touches money is code.

1 Regime classification — implied vs realised vol, trend z-score, expected move

[CODE](#)

2 Directional view — direction, magnitude, horizon, confidence, thesis

[LLM](#)

3	Candidate enumeration — ~37 structures across deltas, widths, sides	CODE
4	Expected-value scoring — $N(d_2)$ under realised vol, tilted by the view	CODE
5	Critic — structural coherence check	LLM
6	Risk gates — 29 deterministic checks	CODE
7	Execution — four-leg mleg order via POST /v2/orders	CODE

This split is also the prompt-injection defence. The agent reads news, which is attacker-influenceable. Because model output is schema-constrained and consumed only as a probability tilt, a fully compromised response can only produce a wrong opinion the EV test must still accept on its merits.

RISK

22 gates, and a breaker that really fires

Sizing uses theoretical max loss, never the observed average — an overnight gap can realise the full amount. The halt path was tested against the live broker, not mocked.

LAYER	WHAT IT ENFORCES	VERIFIED
Structure gates	Short strike $\geq 0.9\sigma$ from spot · credit $\geq 4\%$ of width · positive theta · net delta $\leq 0.35/\text{unit}$	276 tests
Sizing	0.55% of equity per trade · 4% portfolio heat · caps per underlying and per expiry	276 tests
Expectancy	Expected value $\geq 2\%$ of capital at risk, or no trade	276 tests
Circuit breakers	Daily 2% / total 6% drawdown · order-rate cap · kill-switch file	21/21
Broker-side halt	Cancel, flatten, then set suspend_trade on the account itself	real 403
Crash recovery	Exit plan survives an unclean restart; reconciliation finds ghosts and orphans	21/21

A live halt against the broker cancelled orders, flattened the book, set suspend_trade, and Alpaca then **rejected a real order** with 403 "new orders are rejected by user request".

Each surface, for what it actually did

The hackathon requires the MCP server or the CLI. The MCP server is the one wired in; orders go over REST. Options are the only instrument the agent trades.

REST — THE UNATTENDED LOOP

Drives a five-minute cycle: clock check, batched snapshots, option chain with Greeks and IV, then a four-leg mleg submission to POST /v2/orders. Measured at **23 requests per cycle** — 2% of the 200/min budget. Throttled from X-RateLimit=*, not a hard-coded ceiling.

The CLI earned its place differently: inspecting alpaca v0.0.14 is how we learned it defaults to --feed-opra, which 403s on a free account. That fixed every options call we make.

MCP — RESEARCH AND OVERSIGHT

54 tools discovered over stdio JSON-RPC, 12 of them options-specific. Used to inspect the account in natural language and to let the agent look up its own API docs. Session recorded and reproducible.

Alpaca's MCP server wraps every response in a security envelope marking tool output as untrusted, and classifies news and docs as external_text. Our architecture already assumed that.

EVIDENCE

What the filters actually did

We replayed real historical option prices through the live exit logic, then added the agent's filters one at a time. In this window each one improves the outcome — and we then checked whether that holds elsewhere. It does not.

CONFIGURATION	TRADES	WIN%	NET	PROFIT FACTOR	PER TRADE
Naive — sell both sides, every underlying	85	69%	-\$240	0.91	-\$2.82
+ VRP filter — only where implied > realised (drops QQQ)	57	74%	+\$290	1.26	+\$5.09
+ trend filter — never sell calls into an uptrend (as shipped)	36	81%	+\$657	2.52	+\$18.25

The filters do not merely raise the win rate — they remove the specific trades that were losing. Expectancy per trade moves from -\$2.82 to +\$18.25 while the trade count falls from 85 to 36.

Stated honestly — this window only. Re-running the same ladder over three other periods, the filters *hurt*: Aug 2024 turns +\$157 into −\$978, and Mar 2026 gives up \$1,251 of a profitable naive run. The trend and EV filters were tuned with some awareness of this data. Daily bars only, so intraday paths are invisible; strikes by percentage distance because historical Greeks are unavailable; no commissions. **The defensible claim is that the agent refuses to trade when implied vol does not exceed realised — not that these filters generalise.**

Reproduce: `python scripts/filter_ladder.py`. Corroborating: 85 replayed trades across 6 expiry cycles · 21/21 safety tests against the live broker · 123 unit tests · 23 API requests per cycle, 2% of the rate budget.

Where this sits

Two camps exist today. Rules-only bots cannot reason about context; LLM-first agents cannot be trusted with the order ticket.

APPROACH	STRENGTH	WEAKNESS	OPTIONS-NATIVE
Rules-only retail bots preset condor/wheel screeners	Deterministic, auditable, cheap	Blind to regime and context; the same rule fires in a selloff as in a calm week	yes
LLM-first agents prompt + broker key	Reads news and context, adapts quickly	Model picks strikes and sizes, so a bad or injected response reaches the market	rarely
Managed options services subscription signal providers	Human oversight, track record	Not autonomous, not inspectable, slow to act	yes
Options Alpha Agent LLM view, deterministic execution	Context-aware <i>and</i> auditable; every rejection is logged with its reason	Narrow by design — it only sells premium when it is being overpaid for it	only

The differentiator is not the model. It is the boundary: the LLM returns one schema-constrained view, and 22 deterministic gates decide whether anything is sent.

Sizing, with the arithmetic shown

Figures below are an assumption-driven sizing framework, not cited research. The assumptions are stated so a reader can disagree with them precisely.

LAYER	BASIS	ESTIMATE
TAM	Retail options traders on US brokers with an options-approved account	~10M accounts
SAM	Those approved at level 3+ (spreads) and willing to automate via API	~500k
SOM · 3-yr	1% of SAM at a \$25/mo subscription	~\$1.5M ARR

Each number is a ratio applied to the one above it. Replace any ratio and the model updates; nothing here depends on a figure we cannot defend.

REVENUE MODEL

Subscription — tiered by underlyings tracked and capital managed. The agent is broker-agnostic in principle; Alpaca's API is the first integration.

Not performance fees, which require regulatory standing we do not have, and not order-flow monetisation, which would conflict with the risk gates.

WHY IT COULD SUSTAIN

The rejection log is the product. A trader pays to know *why* the agent stood aside, which is the part no signal service provides.

Alpaca paper trading simulation · not investment advice · no live-money path exists in this codebase by design